

Staging Layer Setup

The first step of the ETL implementation was the creation of the `STG\_POLICY` table, which serves as the staging layer for incoming policy data. This table represents the source data that will be transformed and loaded into the integrated layer according to the Source-to-Target Mapping (STTM) requirements.

The staging table contains all source attributes required for the transformation process, including policy information, customer details, state and line-of-business codes, premium amounts, cancellation indicators, and source update timestamps.

A comprehensive set of test records was loaded into the staging table to support validation of the ETL business rules. The test data was intentionally designed to cover both valid and invalid scenarios, allowing verification of transformation logic, data quality rules, lookup validations, defaulting behavior, and error handling.

The test dataset includes:

- Valid policy records representing standard source data.
- Duplicate policy numbers with different `SRC\_UPDATE\_TS` values to validate latest-record selection logic.
- Invalid and malformed date values to test date conversion and validation.
- Valid, invalid, lowercase, and null state codes to support state lookup validation.
- Unknown and null line-of-business codes to test lookup and defaulting behavior.
- Null premium amounts to verify default value assignment.
- Missing first name and last name values to support insured name derivation testing.
- Policy numbers containing leading and trailing spaces to validate trimming logic.
- Various cancellation flag values to support status derivation testing.

The `SRC\_UPDATE\_TS` column was included specifically to support the business requirement that only the most recent record for a policy number should be retained when duplicate records exist in the source.

SQL Implementation - Staging Layer

The following SQL script performs the initial setup of the staging environment for the ETL process.

The script includes:

1. Creation of the Insurance_ETL_CaseStudy database.
2. Creation of the STG_POLICY staging table.
3. Population of test data covering both positive and negative test scenarios.
4. Test records designed to validate ETL transformation, lookup, defaulting, deduplication, and error-handling requirements.

Staging Table Structure

The STG_POLICY table stores raw policy data received from source systems. The SRC_UPDATE_TS column is included to support the business requirement of retaining only the latest record when duplicate policy numbers are encountered.

Test Data Categories

The sample dataset includes records covering the following scenarios:

Category	Purpose
Valid Records	Verify successful processing of standard policy data
Duplicate Policies	Verify latest-record selection using SRC_UPDATE_TS
Invalid Dates	Verify date validation and conversion logic
State Code Variations	Verify state lookup validation and rejection handling
LOB Lookup Scenarios	Verify lookup behavior and default value assignment
Null Premium Amounts	Verify defaulting to 0.00
Name Concatenation Cases	Verify insured name derivation
Policy Number Formatting	Verify trimming of leading/trailing spaces
Status Flag Variations	Verify status derivation logic

SQL Script

```
CREATE DATABASE Insurance_ETL_CaseStudy
```

```
USE Insurance_ETL_CaseStudy
```

```
CREATE TABLE dbo.STG_POLICY (
```

```
    POL_NBR    VARCHAR(20)  NULL,
```

```
    EFF_DT     VARCHAR(8)   NULL,
```

```
    PREM_AMT   DECIMAL(12,2) NULL,
```

```
    ST_CD      CHAR(2)      NULL,
```

```
    LOB_CD     VARCHAR(10)  NULL,
```

```
    LNAME      VARCHAR(40)  NULL,
```

```
    FNAME      VARCHAR(40)  NULL,
```

```
    CANCEL_FLG CHAR(1)      NULL,
```

```
    SRC_UPDATE_TS DATETIME2  NULL
```

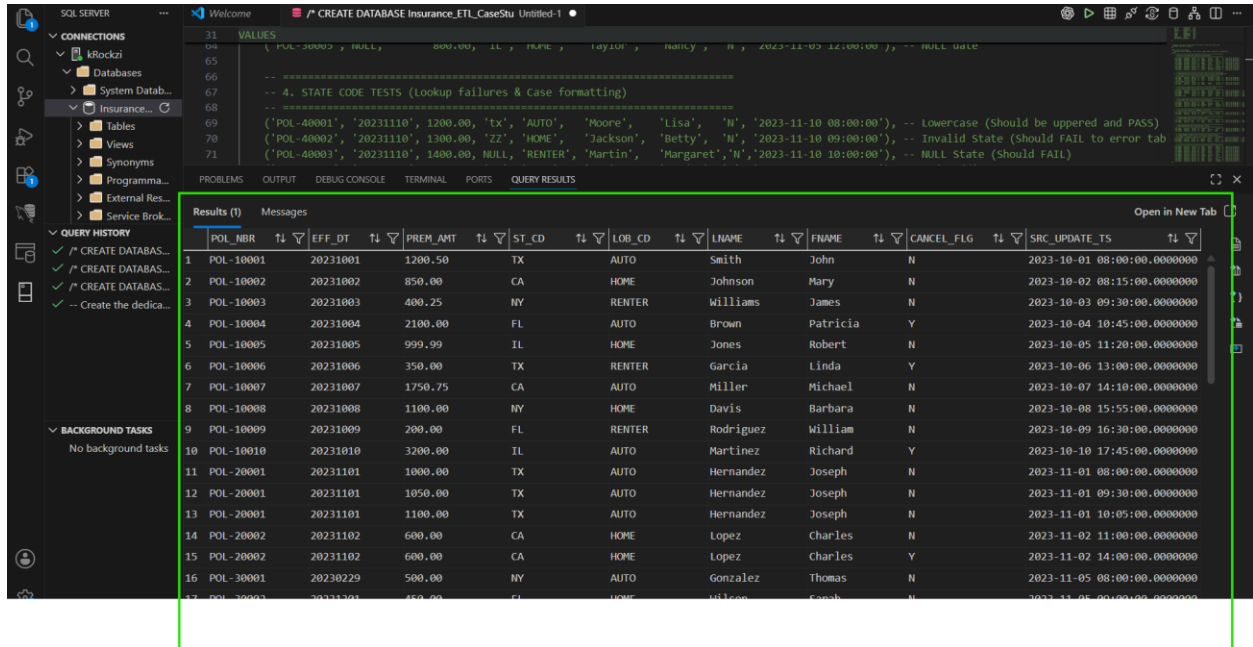
```
);
```

```
INSERT INTO dbo.STG_POLICY (...)
```

```
VALUES (...);
```

Verification Query

```
SELECT *  
  
FROM dbo.STG_POLICY;
```



	POL_NBR	EFF_DT	PREM_AMT	ST_CD	LOB_CD	LNAME	FNAME	CANCEL_FLG	SRC_UPDATE_TS
1	POL-10001	20231001	1200.50	TX	AUTO	Smith	John	N	2023-10-01 08:00:00.0000000
2	POL-10002	20231002	850.00	CA	HOME	Johnson	Mary	N	2023-10-02 08:15:00.0000000
3	POL-10003	20231003	400.25	NY	RENTER	Williams	James	N	2023-10-03 09:30:00.0000000
4	POL-10004	20231004	2100.00	FL	AUTO	Brown	Patricia	Y	2023-10-04 10:45:00.0000000
5	POL-10005	20231005	999.99	IL	HOME	Jones	Robert	N	2023-10-05 11:20:00.0000000
6	POL-10006	20231006	350.00	TX	RENTER	Garcia	Linda	Y	2023-10-06 13:00:00.0000000
7	POL-10007	20231007	1750.75	CA	AUTO	Miller	Michael	N	2023-10-07 14:10:00.0000000
8	POL-10008	20231008	1100.00	NY	HOME	Davis	Barbara	N	2023-10-08 15:55:00.0000000
9	POL-10009	20231009	200.00	FL	RENTER	Rodriguez	William	N	2023-10-09 16:30:00.0000000
10	POL-10010	20231010	3200.00	IL	AUTO	Martinez	Richard	Y	2023-10-10 17:45:00.0000000
11	POL-20001	20231101	1000.00	TX	AUTO	Hernandez	Joseph	N	2023-11-01 08:00:00.0000000
12	POL-20001	20231101	1050.00	TX	AUTO	Hernandez	Joseph	N	2023-11-01 09:30:00.0000000
13	POL-20001	20231101	1100.00	TX	AUTO	Hernandez	Joseph	N	2023-11-01 10:05:00.0000000
14	POL-20002	20231102	600.00	CA	HOME	Lopez	Charles	N	2023-11-02 11:00:00.0000000
15	POL-20002	20231102	600.00	CA	HOME	Lopez	Charles	Y	2023-11-02 14:00:00.0000000
16	POL-30001	20230229	500.00	NY	AUTO	Gonzalez	Thomas	N	2023-11-05 08:00:00.0000000
17	POL-20003	20231101	150.00	FL	HOME	Wilson	Frank	N	2023-11-05 09:00:00.0000000

Reference (Lookup) Tables Setup

To support data validation and business rule implementation, two reference tables were created: `REF\_STATE` and `REF\_LOB`.

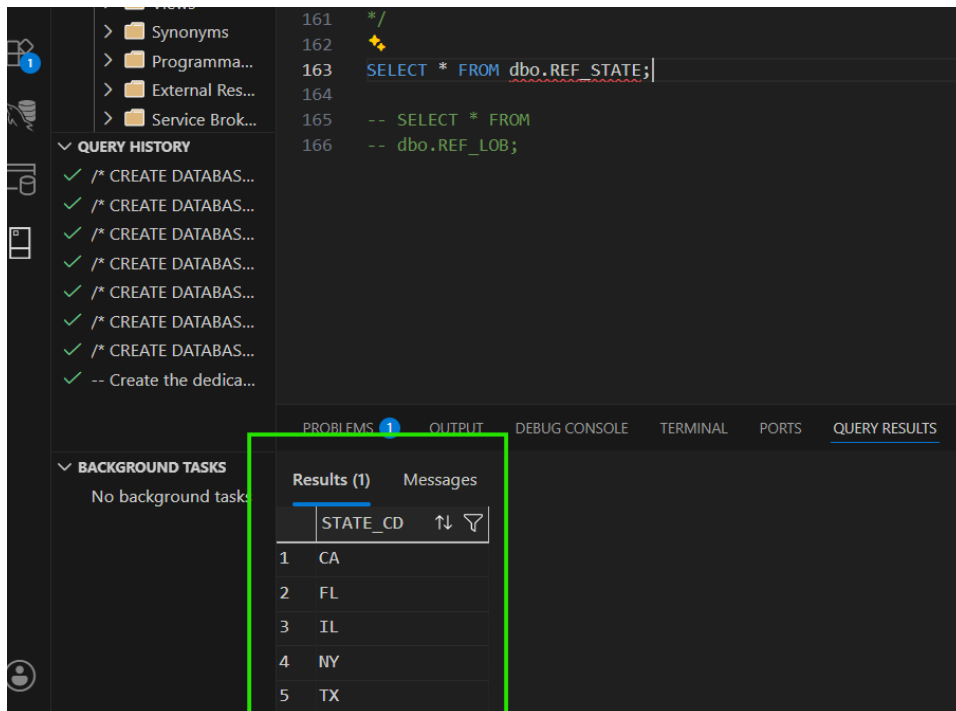
These tables serve as lookup datasets that will be referenced during the ETL transformation process to validate incoming source data and derive target values.

REF_STATE

The `REF\_STATE` table contains the list of valid state codes accepted by the system.

Purpose:

- Validate incoming `ST\_CD` values from `STG\_POLICY`.
- Ensure only recognized state codes are processed.
- Support rejection of records containing invalid or missing state codes.

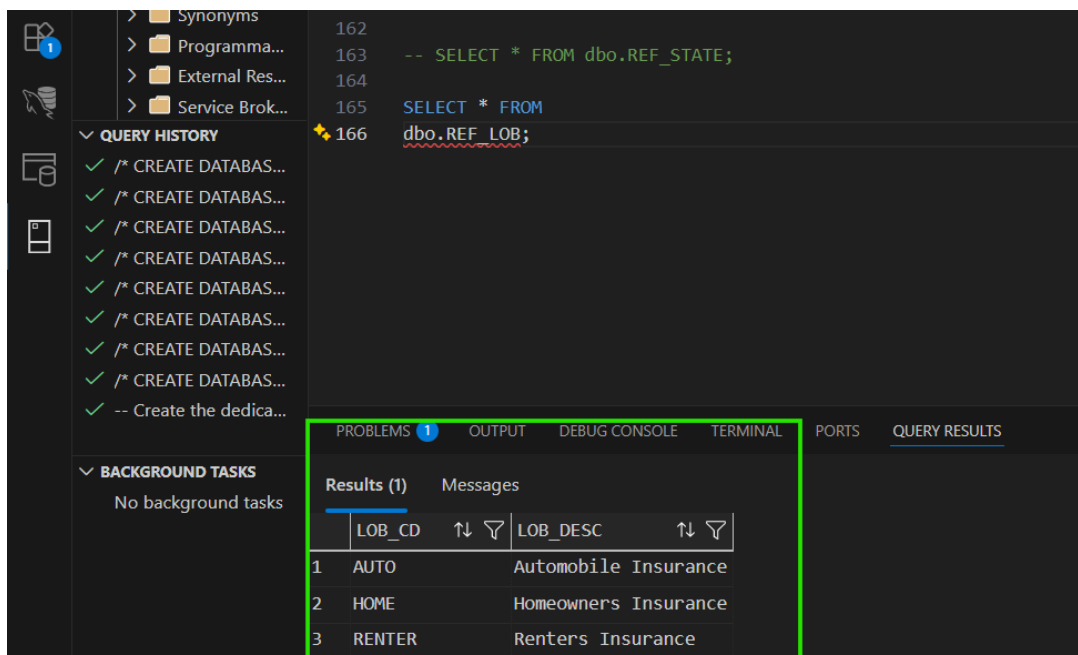


REF_LOB

The `REF\_LOB` table contains valid Line of Business (LOB) codes and their corresponding business descriptions.

Purpose:

- Support lookup business descriptions from source `LOB\_CD` values.
- Standardize Line of Business information during transformation.
- Provide a controlled list of accepted business categories.



Implementation Notes

Both reference tables were populated with a controlled set of values designed to support testing of lookup validations and transformation rules defined in the Source-to-Target Mapping (STTM).

The test data intentionally includes source records containing invalid state codes and unknown Line of Business codes to verify lookup behavior during subsequent ETL processing.

Target and Error Table Setup

Following the creation of the staging and reference tables, the target and error tables were created to support the ETL load process.

ILM_POLICY (Integrated Layer Target Table)

The `ILM\_POLICY` table represents the integrated layer target table defined in the STTM. This table stores the final transformed and validated policy records that successfully pass all business rules and data quality checks.

The table structure reflects the required target mappings and transformations, including:

- Trimmed policy numbers (`POLICY\_NUMBER`)
- Converted effective dates (`EFFECTIVE\_DATE`)
- Standardized premium amounts (`PREMIUM\_AMOUNT`)
- Validated state codes (`STATE\_CODE`)
- Derived line of business descriptions (`LINE\_OF\_BUSINESS`)
- Concatenated insured names (`INSURED\_NAME`)
- Derived policy status (`STATUS`)
- System-generated load timestamp (`LOAD\_TS`)

The `LOAD\_TS` column is automatically populated using `SYSDATETIME()` to provide auditability and track when records are loaded into the integrated layer.

ERROR_POLICY (Rejected Records Table)

The `ERROR\_POLICY` table was created to store records that fail validation during the ETL process.

In accordance with the acceptance criteria, invalid records must not be silently dropped. Instead, all rejected records are captured in the error table together with the reason for rejection.

The table preserves the original staging values in their raw format, allowing failed records to be reviewed, investigated, and reprocessed if necessary.

Additional audit columns were included:

- `REJECT\_REASON` - Stores the business or validation rule that caused the rejection.
- `REJECT\_TS` - Stores the timestamp when the record was rejected.

This design supports the requirement that all rejected records remain traceable and documented throughout the ETL process.

Design Considerations

- `ILM\_POLICY` stores only validated and transformed records.
- `ERROR\_POLICY` stores records that fail validation rules.
- Successful and rejected records are separated to simplify reconciliation and auditing.
- Audit timestamps are automatically generated using `SYSDATETIME()`.
- The error table intentionally allows nullable source columns to ensure invalid source records can be captured without additional load failures.

	name	object_id	principal_id	schema_id	parent_object_id	type	type_desc	create_date	modify_date
1	STG_POLICY	1221579390	NULL	1	0	U	USER_TABLE	2026-06-19 01:14:55.500	2026-06-19 01:14:55.500
2	REF_STATE	1237579447	NULL	1	0	U	USER_TABLE	2026-06-19 01:57:31.237	2026-06-19 01:57:31.237
3	REF_LOB	1269579561	NULL	1	0	U	USER_TABLE	2026-06-19 01:57:31.247	2026-06-19 01:57:31.247
4	ILM_POLICY	1301579675	NULL	1	0	U	USER_TABLE	2026-06-19 03:45:42.107	2026-06-19 03:45:42.107
5	ERROR_POLICY	1333579789	NULL	1	0	U	USER_TABLE	2026-06-19 03:47:13.623	2026-06-19 03:47:13.623

Acceptance Criteria:

Data Validation Assertion and edge case.

1. Grain & Uniqueness Assertions

Objective: Guarantee deterministic deduplication so that only one true record per Policy Number reaches the target.

- Assertion 1 (The Grain): The target table (ILM_POLICY) must contain absolutely no duplicate POLICY_NUMBERS.
- Assertion 2 (The Rule): When multiple records exist in staging for a single policy, the pipeline must isolate and load only the record with the most recent source timestamp.
- Edge Case Addressed (The "Tiebreaker"): If two updates arrive at the exact same millisecond, SQL will normally guess which row to keep, breaking data consistency. This pipeline explicitly handles timestamp collisions by introducing a secondary tie-breaker constraint.
 - Addressed (The NULL Void): In SQL Server, ORDER BY ... DESC forces NULL values to the bottom of the execution order. If a record arrives with a NULL timestamp, the engine will deterministically rank it lower than any record with a valid timestamp, discarding it to protect the target table from un-versioned data.

2. Referential Integrity & Data Routing Assertions

Objective: Ensure invalid categorical data is securely routed to the quarantine zone (ERROR_POLICY), while unmatched non-critical data gracefully defaults to safe values, completely eliminating "silent drops."

- Assertion 1 (Strict Validation): The State Code (ST_CD) must exist in the REF_STATE lookup table. Any unmatched records must be explicitly documented and rejected to the error table.
- Assertion 2 (Graceful Defaulting): The Line of Business (LOB_CD) must be checked against REF_LOB. Unmatched or NULL codes must be successfully loaded into the target table, but overridden with the default value 'UNKNOWN'.
- Edge Cases Addressed (The "Dirty Data" Defense): * State codes often arrive with hidden whitespaces or lowercase letters (e.g., ' tx '). The pipeline forces a TRIM() and UPPER() transformation before attempting the lookup to prevent false-positive rejections.
 - The architecture intentionally utilizes LEFT JOINS for all reference lookups. Using an INNER JOIN would cause invalid states to completely vanish from the pipeline execution (a silent drop).

3. Data Typing & Formatting Assertions

Objective: Prevent fatal pipeline crashes during raw string conversions and ensure strict mathematical accuracy for downstream financial reporting.

- Assertion 1 (Safe Date Casting): The EFFECTIVE_DATE must securely cast from a raw string to a standard calendar date. Any failure to cast must be intercepted and routed to the error table rather than causing a pipeline failure.
- Assertion 2 (Financial Precision & Defaults): The PREMIUM_AMOUNT must enforce a rigid decimal precision. If the source system fails to provide a premium (NULL), the pipeline must mathematically default the value to 0.00 to protect downstream aggregation metrics.
- Edge Cases Addressed (The System Crashers): * Invalid calendar dates (e.g., February 29th on a non-leap year or month 13) naturally crash databases during standard ingestion. The pipeline uses defensive casting to evaluate the date's validity before attempting a hard insert.
 - Empty strings or garbage alphabetical characters arriving in the date field are explicitly flagged as INVALID_DATE_FORMAT and quarantined.

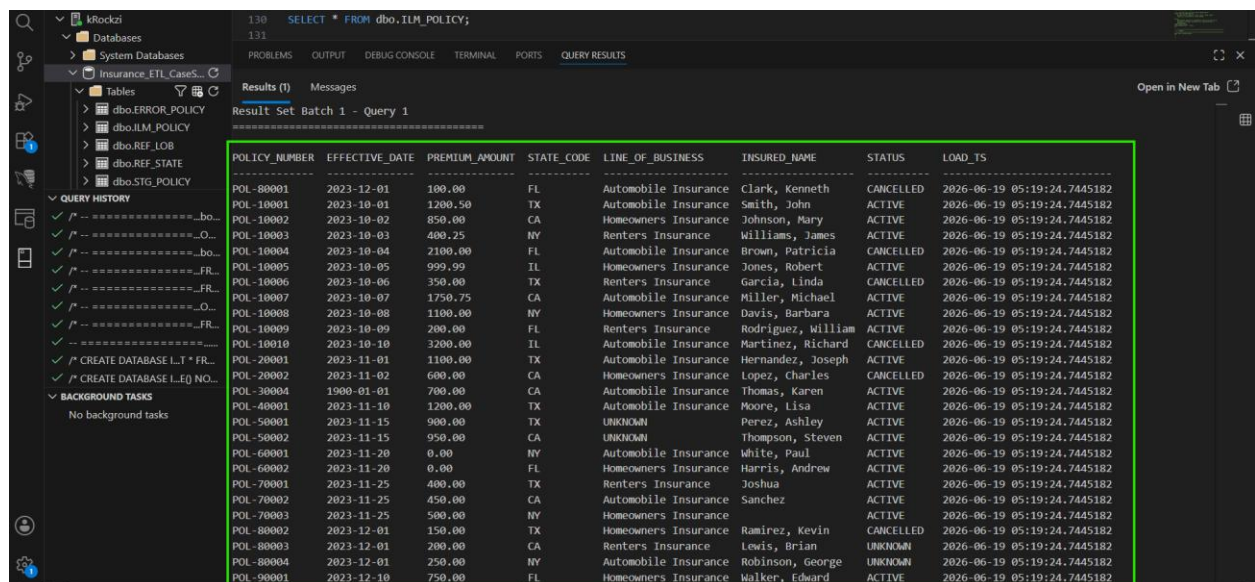
4. Business Logic & Cleansing Assertions

Objective: Standardize free-text formatting, gracefully handle missing human-entry data, and decode operational flags into human-readable dimensions.

- Assertion 1 (String Cleansing): The POLICY_NUMBER must be stripped of all leading and trailing whitespaces before entering the Integrated Layer to ensure accurate reporting aggregations.
- Assertion 2 (Safe Concatenation): The INSURED_NAME must combine the Last Name and First Name fields. If either name is missing, the pipeline must preserve the existing name rather than nullifying the entire field.
- Assertion 3 (Status Decoding): The raw CANCEL_FLG must be strictly mapped. A 'Y' translates to 'CANCELLED', an 'N' translates to 'ACTIVE', and any other unmapped or missing value safely defaults to 'UNKNOWN'.
- Edge Cases Addressed (The Null-Wipeout Defense): * In database architecture, concatenating a valid string with a NULL yields a NULL output. The pipeline uses conditional logic to evaluate the presence of the first and last names independently, ensuring that a missing first name does not accidentally erase a perfectly valid last name from the final target table.

Final table (Post Transformation) :

```
SELECT * FROM dbo.ILM_POLICY;
```



POLICY_NUMBER	EFFECTIVE_DATE	PREMIUM_AMOUNT	STATE_CODE	LINE_OF_BUSINESS	INSURED_NAME	STATUS	LOAD_TS
POL-80001	2023-12-01	100.00	FL	Automobile Insurance	Clark, Kenneth	CANCELLED	2026-06-19 05:19:24.7445182
POL-10001	2023-10-01	1200.50	TX	Automobile Insurance	Smith, John	ACTIVE	2026-06-19 05:19:24.7445182
POL-10002	2023-10-02	850.00	CA	Homeowners Insurance	Johnson, Mary	ACTIVE	2026-06-19 05:19:24.7445182
POL-10003	2023-10-03	400.25	NY	Renters Insurance	Williams, James	ACTIVE	2026-06-19 05:19:24.7445182
POL-10004	2023-10-04	2100.00	FL	Automobile Insurance	Brown, Patricia	CANCELLED	2026-06-19 05:19:24.7445182
POL-10005	2023-10-05	999.99	IL	Homeowners Insurance	Jones, Robert	ACTIVE	2026-06-19 05:19:24.7445182
POL-10006	2023-10-06	350.00	TX	Renters Insurance	Garcia, Linda	CANCELLED	2026-06-19 05:19:24.7445182
POL-10007	2023-10-07	1750.75	CA	Automobile Insurance	Miller, Michael	ACTIVE	2026-06-19 05:19:24.7445182
POL-10008	2023-10-08	1100.00	NY	Homeowners Insurance	Davis, Barbara	ACTIVE	2026-06-19 05:19:24.7445182
POL-10009	2023-10-09	200.00	FL	Renters Insurance	Rodriguez, William	ACTIVE	2026-06-19 05:19:24.7445182
POL-10010	2023-10-10	3200.00	IL	Automobile Insurance	Martinez, Richard	CANCELLED	2026-06-19 05:19:24.7445182
POL-20001	2023-11-01	1100.00	TX	Automobile Insurance	Hernandez, Joseph	ACTIVE	2026-06-19 05:19:24.7445182
POL-20002	2023-11-02	600.00	CA	Homeowners Insurance	Lopez, Charles	CANCELLED	2026-06-19 05:19:24.7445182
POL-30004	1980-01-01	700.00	CA	Automobile Insurance	Thomas, Karen	ACTIVE	2026-06-19 05:19:24.7445182
POL-40001	2023-11-10	1200.00	TX	Automobile Insurance	Moore, Lisa	ACTIVE	2026-06-19 05:19:24.7445182
POL-50001	2023-11-15	900.00	TX	UNKNOWN	Perez, Ashley	ACTIVE	2026-06-19 05:19:24.7445182
POL-50002	2023-11-15	950.00	CA	UNKNOWN	Thompson, Steven	ACTIVE	2026-06-19 05:19:24.7445182
POL-60001	2023-11-20	0.00	NY	Automobile Insurance	White, Paul	ACTIVE	2026-06-19 05:19:24.7445182
POL-60002	2023-11-20	0.00	FL	Homeowners Insurance	Harris, Andrew	ACTIVE	2026-06-19 05:19:24.7445182
POL-70001	2023-11-25	400.00	TX	Renters Insurance	Sanchez	ACTIVE	2026-06-19 05:19:24.7445182
POL-70002	2023-11-25	450.00	CA	Automobile Insurance	Joshua	ACTIVE	2026-06-19 05:19:24.7445182
POL-70003	2023-11-25	580.00	NY	Homeowners Insurance		ACTIVE	2026-06-19 05:19:24.7445182
POL-80002	2023-12-01	150.00	TX	Homeowners Insurance	Ramirez, Kevin	CANCELLED	2026-06-19 05:19:24.7445182
POL-80003	2023-12-01	200.00	CA	Renters Insurance	Lewis, Brian	UNKNOWN	2026-06-19 05:19:24.7445182
POL-80004	2023-12-01	250.00	NY	Automobile Insurance	Robinson, George	UNKNOWN	2026-06-19 05:19:24.7445182
POL-90001	2023-12-10	750.00	FL	Homeowners Insurance	Walker, Edward	ACTIVE	2026-06-19 05:19:24.7445182

Error Table:

```
SELECT * FROM dbo.ERROR_POLICY;
```

POL_NBR	EFF_DT	PREM_AMT	ST_CD	LOB_CD	LNAME	FNAME	CANCEL_FLG	SRC_UPDATE_TS	REJECT_REASON	REJECT_TS
POL-30001	20230229	500.00	NY	AUTO	Gonzalez	Thomas	N	2023-11-05 08:00:00.0000000	INVALID_OR_MISSING_DATE	2026-06-19 05:19:24.7445182
POL-30002	20231301	450.00	FL	HOME	Wilson	Sarah	N	2023-11-05 09:00:00.0000000	INVALID_OR_MISSING_DATE	2026-06-19 05:19:24.7445182
POL-30003	ABCDEFQH	300.00	TX	RENTER	Anderson	Matthew	N	2023-11-05 10:00:00.0000000	INVALID_OR_MISSING_DATE	2026-06-19 05:19:24.7445182
POL-30005	NULL	800.00	IL	HOME	Taylor	Nancy	N	2023-11-05 12:00:00.0000000	INVALID_OR_MISSING_DATE	2026-06-19 05:19:24.7445182
POL-40002	20231110	1300.00	ZZ	HOME	Jackson	Betty	N	2023-11-10 09:00:00.0000000	INVALID_OR_MISSING_STATE_CODE	2026-06-19 05:19:24.7445182
POL-40003	20231110	1400.00	NULL	RENTER	Martin	Margaret	N	2023-11-10 10:00:00.0000000	INVALID_OR_MISSING_STATE_CODE	2026-06-19 05:19:24.7445182
POL-40004	20231110	1500.00	C	AUTO	Lee	Sandra	N	2023-11-10 11:00:00.0000000	INVALID_OR_MISSING_STATE_CODE	2026-06-19 05:19:24.7445182

Task 2:

Reconciliation:

Note : The following reconciliation query is designed to be run after the master ETL pipeline .

```
WITH Source_Distinct AS (  
    SELECT COUNT(DISTINCT POL_NBR) AS Total_Inbound_Policies  
    FROM dbo.STG_POLICY  
)  
  
SELECT  
    Total_Inbound_Policies,  
    (SELECT COUNT(*) FROM dbo.ILM_POLICY) AS Target_Loaded,  
    (SELECT COUNT(*) FROM dbo.ERROR_POLICY) AS Error_Rejected,  
  
    Total_Inbound_Policies - (  
        (SELECT COUNT(*) FROM dbo.ILM_POLICY) +  
        (SELECT COUNT(*) FROM dbo.ERROR_POLICY)
```

) AS Record_Variance

FROM Source_Distinct;

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Connections' pane shows the 'Insurance_ETL_Case...' database. The 'Query History' pane lists several queries, with the most recent one highlighted. The main window shows a T-SQL query script. The query includes a comment about counting unique policies, a CTE named 'Source_Distinct', and a SELECT statement that calculates 'Record_Variance' as the difference between 'Total_Inbound_Policies' and the sum of 'Target_Loaded' and 'Error_Rejected'. The query concludes with 'FROM Source_Distinct;' and 'GO'. Below the query editor, the 'Query Results' pane shows a table with one row of data.

```
123 DROP TABLE #TempRoutedData;
124 GO
125
126
127 WITH Source_Distinct AS (
128     -- Count the unique policies arriving from staging
129     SELECT COUNT(DISTINCT POL_NBR) AS Total_Inbound_Policies
130     FROM dbo.STG_POLICY
131 )
132 SELECT
133     Total_Inbound_Policies,
134     (SELECT COUNT(*) FROM dbo.ILM_POLICY) AS Target_Loaded,
135     (SELECT COUNT(*) FROM dbo.ERROR_POLICY) AS Error_Rejected,
136
137     -- The Variance MUST be 0. If it is not 0, a silent drop occurred.
138     Total_Inbound_Policies - (
139         (SELECT COUNT(*) FROM dbo.ILM_POLICY) +
140         (SELECT COUNT(*) FROM dbo.ERROR_POLICY)
141     ) AS Record_Variance
142 FROM Source_Distinct;
143 GO
144
145
146
147
```

Results (1)	Messages		
Total_Inbound_Policies	Target_Loaded	Error_Rejected	Record_Variance
1 44	37	7	0